==========

==========

CHAPTER 10 — BREAK-BEFORE-MAKE (BBM)

==========

==========

A maximum-detail, RTOS/QNX-style technical breakdown of ARMv8's Break-Before-Make rule, why it exists, how it prevents TLB corruption, and the exact sequences required for safe page-table updates.

****

****

10.1 — Introduction to Break-Before-Make (BBM)

****

****

Break-Before-Make (BBM) is one of the most critical MMU safety rules in ARMv8.

BBM requires:

Before replacing a valid page-table descriptor with another valid descriptor, the old descriptor must be invalidated and fully removed from the TLB.

This rule prevents:

- TLB aliasing
- stale translations
- mixed attributes
- mixed permissions
- undefined behavior
- silent memory corruption

BBM applies to **both Stage-1 and Stage-2** translations.

****

****

10.2 — Why BBM Exists

****

****

ARMv8 TLBs aggressively cache:

- block mappings
- page mappings
- permissions
- memory attributes
- shareability
- execute permissions

If a descriptor is replaced without invalidating the old one:

- TLB may contain both old and new mappings

- CPU may use stale attributes
- CPU may use stale permissions
- walk cache may contain stale intermediate descriptors
- speculative execution may use stale mappings
- multiple cores may observe inconsistent translations

Teaching Note

BBM prevents **TLB aliasing**, the most dangerous MMU failure mode.

****

****

10.3 — When BBM Is Required

****

****

BBM is required whenever a descriptor changes type or meaning.

Required for:

- block → page
- page → block
- block → table
- table → block
- changing physical address
- changing memory attributes
- changing permissions
- changing shareability
- changing execute permissions
- changing AF
- changing AttrIndx

Not required for:

- invalid → valid
- valid → invalid

Implementation Note

BBM is required even if the new descriptor maps the same PA.

****

****

10.4 — Descriptor Type Changes Requiring BBM

****

****

Case 1 — Block → Page

Example: splitting a 2 MB block into 512 × 4 KB pages.

Case 2 — Page → Block

Example: merging pages into a block for performance.

Case 3 — Block → Table

Example: replacing a block with a table to increase granularity.

Case 4 — Table → Block

Example: collapsing a table into a block.

Teaching Note

Any change in descriptor type requires BBM.

****

****

10.5 — Attribute Changes Requiring BBM

****

****

Changing memory attributes requires BBM:

- Normal WB → Device-nGnRnE
- Device → Normal
- Normal NC → Normal WB
- Write-Through → Write-Back

Why?

TLB entries store:

- merged Stage-1 + Stage-2 attributes
- cacheability
- ordering rules

Stale attributes cause:

- cache corruption
- device malfunction
- ordering violations

****

****

10.6 — Permission Changes Requiring BBM

****

****

Changing AP or S2AP requires BBM:

- read/write → read-only
- read-only → no access
- executable → non-exec
- non-exec → executable

Why?

TLB entries store:

- merged permissions
- execute permissions

Stale permissions cause:

- security violations
- kernel/user privilege leaks
- hypervisor privilege leaks

****

****

10.7 — Physical Address Changes Requiring BBM

****

****

Changing PA_base requires BBM:

- remapping memory
- moving pages
- swapping pages
- ballooning
- live migration
- memory overcommit

Why?

TLB may contain stale PA → memory corruption.

****

****

10.8 — Full BBM Sequence (Correct Order)

****

****

The correct BBM sequence is:

Step 1 — Break

Invalidate old descriptor:

```
descriptor = INVALID
```

Step 2 — DSB SY

Ensure descriptor write is visible:

```
DSB SY
```

Step 3 — TLBI

Invalidate stale TLB entries:

```
TLBI VAE1, xN
```

Step 4 — DSB SY

Ensure TLBI completes:

```
DSB SY
```

Step 5 — Make

Install new descriptor:

```
descriptor = NEW_DESCRIPTOR
```

Step 6 — DSB SY

Ensure new descriptor visible:

```
DSB SY
```

Step 7 — ISB

Flush pipeline:

```
ISB
```

****

****

10.9 — BBM Sequence (Table → Block)

****

****

Example: collapsing a table into a block.

Step 1 — Invalidate table descriptor

Step 2 — DSB SY

Step 3 — TLBI

Step 4 — DSB SY

Step 5 — Install block descriptor

Step 6 — DSB SY

Step 7 — ISB

Teaching Note

Walk cache must be invalidated via TLBI.

****

****

10.10 — BBM Sequence (Block → Page)

****

****

Example: splitting a block into pages.

Step 1 — Invalidate block descriptor

Step 2 — DSB SY

Step 3 — TLBI

Step 4 — DSB SY

Step 5 — Install table descriptor

Step 6 — Install page descriptors

Step 7 — DSB SY

Step 8 — ISB

Implementation Note

All page descriptors must be installed before enabling access.

****

****

10.11 — BBM and Walk Cache

****

****

Walk cache stores:

- intermediate descriptors
- table pointers
- partial walk results

BBM ensures:

- walk cache does not contain stale descriptors
- walk cache does not contain stale table pointers

Teaching Note

TLBI invalidates walk cache entries.

****

****

10.12 — BBM and TLB Aliasing

****

****

TLB aliasing occurs when:

- old descriptor maps PA1
- new descriptor maps PA2
- TLB contains both mappings

This causes:

- inconsistent memory access
- unpredictable behavior
- silent corruption

Implementation Note

BBM prevents aliasing.

****

****

10.13 — BBM and Speculation

****

****

Speculative execution may:

- read stale descriptors
- fill TLB entries
- use stale mappings

BBM ensures:

- stale descriptors invalidated
- stale TLB entries removed
- pipeline flushed

Teaching Note

ISB is required to flush speculative state.

****

****

10.14 — BBM and SMP Systems

****

****

Multiple cores may:

- observe stale descriptors
- observe stale TLB entries
- observe stale walk cache entries

BBM ensures:

- ordering across cores
- visibility across cores
- correctness of shared memory

Implementation Note

DSB SY ensures global visibility.

****

****

10.15 — BBM and Hypervisor Interaction

****

****

Hypervisor uses BBM for:

- Stage-2 remapping
- VM migration
- memory overcommit
- nested virtualization
- device memory remapping

Teaching Note

Stage-2 BBM is identical to Stage-1 BBM.

****

****

10.16 — BBM and Nested Virtualization

****

****

Nested virtualization requires:

- nested BBM
- nested TLBI
- nested descriptor updates
- nested barrier sequences

Implementation Note

Nested BBM must be carefully ordered to avoid deadlocks.

****

****

10.17 — BBM and QNX/RTOS Behavior

****

****

QNX uses BBM for:

- deterministic memory updates
- partition isolation
- safe page table updates
- SMP correctness
- device driver safety

Teaching Note

RTOS environments require strict BBM compliance.

****

****

10.18 — Common BBM Mistakes

****

****

Mistake 1 — Installing new descriptor before invalidating old

Causes aliasing.

Mistake 2 — Missing TLBI

Stale TLB entries remain.

Mistake 3 — Missing DSB

Descriptor writes not visible.

Mistake 4 — Missing ISB

Pipeline uses stale TTBR/TCR.

Mistake 5 — Changing attributes without BBM

Causes cache corruption.

****

****

10.19 — Full BBM Example

****

****

Example: Changing memory type from Normal WB → Device-nGnRnE

```
STR x0, [descriptor]        // write INVALID descriptor
DSB SY                      // ensure visibility
TLBI VAE1, x1               // invalidate TLB entry
DSB SY                      // ensure TLBI complete
STR x2, [descriptor]        // write new descriptor
DSB SY                      // ensure visibility
ISB                         // flush pipeline
```

****

****

10.20 — Chapter Summary

****

****

This chapter covered:

- BBM purpose
- when BBM is required
- descriptor type changes
- attribute changes
- permission changes
- physical address changes
- full BBM sequences
- walk cache interaction
- TLB aliasing
- speculation
- SMP systems
- hypervisor interaction
- nested virtualization
- QNX/RTOS considerations
- common mistakes
- full examples